MODULE 23 · DOMAIN 5 - DATA COLLABORATION

Time Travel, Fail-safe and zero-copy cloning

Domain 5 begins. Three features built on one idea: micro-partitions are immutable.

DOMAIN 5 of 5 WEIGHT 10% MODULE 23 of 31

admin@sthithapragna.com

Because micro-partitions are **immutable**, the old version of your data still exists. These three features are what you can do with it.

The retention grid

TABLE TYPE	TIME TRAVEL	FAIL-SAFE
Permanent, Standard Edition	0 or 1 day. Default 1	7 days
Permanent, Enterprise or higher	0 to 90 days. Default 1	7 days
Transient	0 or 1 day	None
Temporary	0 or 1 day	None

ON THE EXAM

90 days is Enterprise and permanent. Both conditions must hold, which is why Domain 1's drill closed on exactly that.

Time Travel

- Query data as it was, using **AT** or **BEFORE** with **TIMESTAMP**, **OFFSET** in seconds, or **STATEMENT** with a query ID.
- **STATEMENT is the precise one.** Point at the query ID of the statement that caused the damage and read the state just before it.
- Works in a **SELECT**, in **CREATE … CLONE**, and with **UNDROP**.
- **UNDROP** restores a dropped table, schema or database, provided you are still inside the window. It restores the most recent version.
- Set with **DATA_RETENTION_TIME_IN_DAYS**, which can be set at account, database, schema or table level.

> ON THE EXAM
>
> Setting retention to 0 disables Time Travel entirely, and therefore UNDROP too. That is a deliberate cost decision, not an accident.

Time Travel or Fail-safe?

TIME TRAVEL

Yours

- 0 to 90 days, depending on edition and table type
- You query it yourself with ordinary SQL
- Immediate. AT, BEFORE, UNDROP, CLONE
- You choose the retention, and pay for it

FAIL-SAFE

Snowflake's

- A further 7 days after Time Travel expires
- Only Snowflake can recover from it, via support
- Best-effort, and may take hours or days
- Not configurable, and permanent objects only

THE TIEBREAKER

Fail-safe is a last resort, not a backup. If a scenario needs self-service recovery, Fail-safe is always the wrong answer.

Zero-copy cloning

- **A metadata operation.** The clone points at the same micro-partitions. No data is copied and it is near-instant.
- **No storage cost until it diverges.** Storage grows only as rows are added, deleted or modified in either object.
- The clone is **fully independent**. Changes to one do not affect the other.
- Clone a **table, schema or database**, and with **AT or BEFORE** to clone it as it was at a point in time.
- A clone's **history begins at the moment of cloning**. You cannot Time Travel a clone back to before it existed.

> ON THE EXAM
>
> Cloning a 40 TB production database for testing takes seconds and costs nothing on day one. The cost arrives as the clone diverges.

What a clone does not inherit

- **Load history.** A cloned table has none, so **files already loaded into the source can be loaded again into the clone.**
- **Automatic Clustering starts suspended** on the clone, even if it is active on the source. The clustering key itself is replicated.
- **Cloned tasks are created suspended** and must be resumed.
- **Grants.** CREATE … CLONE does not copy grants for most objects. Use **COPY GRANTS**.
- **Unconsumed records in cloned streams are inaccessible**, consistent with how Time Travel behaves for tables.

> ON THE EXAM
>
> The load history one causes real duplication. A team clones prod to dev, re-runs the pipeline, and loads the same files twice.

The container subtlety with grants

- When you clone a **database or schema**, the clone **inherits granted privileges on the clones of all child objects**.
- But **the clone of the container itself does not inherit** the privileges granted on the source container.
- So the cloned tables keep their grants, and nobody can reach them, because the schema grant is missing.
- Clones also inherit any **future grants** defined for that object type in the schema.

> ON THE EXAM
>
> This is why a cloned database looks correctly permissioned and is still inaccessible. Recall module 8: you need USAGE on the container too.

How this module gets tested

THE REQUIREMENT	THE ANSWER
Undo a bad UPDATE from twenty minutes ago	Time Travel. AT or BEFORE with STATEMENT or OFFSET
Recover a table somebody dropped this morning	UNDROP, if still inside the retention window
Recover data from six weeks ago on Standard Edition	Not possible. Standard permanent tops out at 1 day
Give the test team a copy of production today	Zero-copy clone. Near-instant, no storage until it diverges
Recover after Time Travel has expired	Fail-safe, via Snowflake Support. Permanent objects only
The cloned pipeline loaded everything twice	A clone has no load history

ON THE EXAM

Six shapes. The third and the fifth are the ones candidates get wrong, in opposite directions.

> A team clones the production database into a development database so testers can work against realistic data. The development environment runs the same nightly COPY pipeline against the same external stage, which still holds the last thirty days of files.
>
> The next morning, several tables in the development clone hold roughly double the expected rows. The pipeline reported no errors.

What happened?

A The clone shares micro-partitions with the source, so rows loaded into production appeared in the clone as well.

B FORCE = TRUE must have been set on the COPY statement, overriding load metadata.

C A cloned table does not inherit load history, so files already loaded into the source were loaded again.

D Time Travel on the clone replayed the previous day's DML into the new tables.

◆ PAUSE AND DECIDE

What happened?

A The clone shares partitions, so production rows appeared.

Sharing partitions is how cloning works, and the clone is fully independent from the moment it is created. New production rows do not appear in it.

B FORCE = TRUE was set.

Would produce the same symptom, and the scenario says the same pipeline ran unchanged. The clone explains it without anyone changing anything.

C A clone has no load history, so the files loaded again.

Documented. The clone's load metadata is empty, so every file in the stage looks new to it.

D Time Travel replayed DML.

Time Travel lets you read past states. It never replays anything into a table.

THE SIGNAL

Modules 14 and 23 together. The 64-day load metadata belongs to the source table, and the clone starts with none of it.

> A permanent table on Enterprise Edition has DATA_RETENTION_TIME_IN_DAYS set to 3. Eleven days ago a bulk delete removed several million rows in error, and nobody noticed until an audit this week.
>
> A team lead proposes querying the table with AT(OFFSET => -950400) to read the state before the delete.

Will that work, and what are the options?

A. No. Time Travel expired after 3 days. The only remaining route is Fail-safe, via Snowflake Support.

B. Yes. OFFSET is not bounded by DATA_RETENTION_TIME_IN_DAYS, only TIMESTAMP is.

C. No, but raising DATA_RETENTION_TIME_IN_DAYS to 90 now will restore access to the older data.

D. Yes, because Enterprise Edition provides 90 days regardless of the retention setting.

◆ PAUSE AND DECIDE

Will that work, and what are the options?

A No. Fail-safe, via Snowflake Support.

Retention was 3 days, so day 11 is well past it. Fail-safe covers 7 days beyond that, and it is a best-effort support request.

B OFFSET is not bounded.

All three forms of AT and BEFORE are bounded by the retention period. The argument type does not change the window.

C Raise retention now.

Changing the setting affects data going forward. It cannot resurrect history already outside the previous window.

D Enterprise gives 90 days regardless.

Enterprise raises the maximum you may configure. The default is still 1 day, and this table was set to 3.

THE SIGNAL

Enterprise permits up to 90 days. It does not provide them. Somebody has to set the parameter, and here they set 3.

> An analyst ran an UPDATE without a WHERE clause forty minutes ago, overwriting a status column across an entire 8 TB table. Other teams have written legitimate rows to the table since, which must be preserved.
>
> The account is on Enterprise Edition with 7 days of retention. The analyst has the query ID of the bad UPDATE.

What is the cleanest recovery?

A. UNDROP the table, which restores the most recent version before the UPDATE.

B. Raise DATA_RETENTION_TIME_IN_DAYS to 90, then query the table as it was.

C. Contact Snowflake Support to recover from Fail-safe, since the data has been overwritten.

D. CLONE the table BEFORE(STATEMENT => the bad query ID), then merge the correct status values back.

◆ PAUSE AND DECIDE

What is the cleanest recovery?

A. UNDROP the table.

UNDROP restores a dropped object. Nothing was dropped here; the table exists and holds wrong values.

B. Raise retention to 90 first.

Unnecessary. The event was forty minutes ago and 7 days of retention already covers it comfortably.

C. Fail-safe, via Support.

Fail-safe applies only after Time Travel expires. Time Travel is available and immediate here.

D. CLONE BEFORE(STATEMENT), then merge back.

STATEMENT points precisely at the bad UPDATE, the clone is instant and free, and merging preserves the legitimate writes since.

THE SIGNAL

'Other teams have written since' rules out any full restore. You need the old values beside the current table, not instead of it.

> An administrator clones a production database to create a staging environment, then tells the QA team it is ready. The QA role held SELECT on every table in production.
>
> QA reports that they cannot see the staging database at all. Checking SHOW GRANTS on the cloned tables shows their SELECT grants are intact.

Why can they not reach the data?

A Cloned objects require COPY GRANTS, without which no grants are inherited at any level.

B The clone of the container does not inherit privileges granted on the source container. USAGE on the database and schema is missing.

C Clones are owned by the cloning role and cannot be granted to other roles until ownership is transferred.

D The QA role needs its secondary roles enabled before grants on cloned objects take effect.

◆ PAUSE AND DECIDE

Why can they not reach the data?

A COPY GRANTS needed, or nothing is inherited.

True for most single objects, and the scenario says the table grants ARE intact. Cloning a database does propagate child grants.

B The container clone inherits nothing. USAGE is missing.

Documented: child object grants are inherited, but the cloned database and schema do not inherit grants on their sources.

C Ownership must be transferred first.

The cloning role owns the clone, and ownership does not prevent granting. Nothing needs transferring.

D Secondary roles must be enabled.

That applies to privileges granted directly to a user under UBAC. These were granted to a role.

THE SIGNAL

Modules 8 and 23. The tables are correctly permissioned and unreachable, because containment still applies to a clone.

What to carry into the next module

- **All three features exist because micro-partitions are immutable.** The old version is still there.
- **Time Travel is yours, Fail-safe is Snowflake's.** 7 days, permanent objects only, support request, best-effort.
- **90 days needs Enterprise AND permanent AND somebody to set the parameter.** The default is 1 day.
- **AT or BEFORE with TIMESTAMP, OFFSET or STATEMENT.** STATEMENT is the precise one. Cloning accepts them too.
- **A clone has no load history**, starts with clustering and tasks suspended, and its container inherits no grants.

> ON THE EXAM
>
> Next: secure sharing, listings, clean rooms and native apps. Sharing without copying, built on the same immutability.